Workshop 1: Designing a System That Scales to Millions of Users — Final Report

Course: Fundamentals of System Security (COMSM0122)

1. Executive Summary & Problem Scope

Modern distributed applications need to grow smoothly from serving a handful of users to handling millions of concurrent requests — all while staying cost-effective, highly available, and resilient against denial-of-service (DoS) attacks. This workshop set out to design, implement, and stress-test an incremental scaling architecture using a local Node.js testbed.

The system core handles three main use cases:

- Processing read/write user requests carrying variable data payloads

- Providing a session service that processes and securely stores request payloads in backend datastores

- Scaling horizontally and vertically to withstand high-density traffic intervals and adversarial threat vectors

2. Task A: Scalable System Architecture & Load Simulation

Architecture Progression Tiers

Tens of Users: Handled with a baseline setup — a minimal server pool and a single datastore, managed by a Round-Robin load balancer.

Hundreds to Thousands of Users: Scaled horizontally by increasing server replicas (up to 100), switching to a Least-Connections routing algorithm to bypass slow or saturated nodes, and sharding across multiple datastores.

Millions of Users: Implemented robust session bounds, optimized processing times, and distributed storage clusters to prevent bottlenecks.

Load-Balancing Algorithm Comparison

| Algorithm | Behaviour | Strengths & Weaknesses |

|---|---|---|

| Round Robin (RR) | Distributes requests sequentially | Effective under even network loads, but performs poorly when server processing times vary, leading to head-of-line blocking |

| Least Connections (LC) | Dynamically tracks active sessions per server and routes to the node with the fewest open connections | Highly effective for handling uneven request sizes and processing speeds |

| Consistent Hashing (CH) | Ideal for caching and sharded datastore affinity | Vulnerable to hash ring hotspots if traffic clusters heavily around specific keys |

Simulation Results Summary (RequestSimulation.js)

- High Intervals (500ms – 200ms): Maintained a 100% success rate, as servers and datastores cleared requests well within processing bounds.

- Low Intervals / Burst Conditions (20ms – 1ms): Success rates dropped sharply due to session exhaustion (`MaxNoSessionsPerServer`) and datastore storage capacity limits (`MaxDatastoreSpace`).

3. Task B: Resilience & Abusive Traffic Defences

B.1 Fault Tolerance & Automatic Recovery

- Active Health Checks: Integrated background probes checking server and datastore heartbeats every 50ms.

- Failover Routing: Caught mid-flight execution errors during server crashes, incremented failover retry metrics, and transparently rerouted requests to surviving healthy replicas.

- Node Recovery: Automatically quarantined unhealthy nodes and restored them to active rotation once health checks passed, ensuring minimal downtime.B.2 Abusive Traffic Defences & Threat Model

Threat Model: Identified and mitigated key attack vectors, including volumetric HTTP bursts (exhausting session slots), oversized payload attacks (exhausting datastore storage), and slow-processing resource hogging.

Defensive Controls Implemented:

- Payload Size Validation: Dropped requests exceeding `MaxSizeRequestPayload` immediately (413 Payload Too Large).

- Sliding-Window Rate Limiting: Blocked high-rate bot floods (429 Too Many Requests).

- Request Timeouts: Enforced hard timeouts via `Promise.race()` to abort hanging requests (408 Request Timeout).